Project Design Phase
Solution Architecture

Date	13 February 2026
Team ID	LTVIP2026TMIDS81332
Project Name	flightfinder: navigating your air travel options
Maximum Marks	4 Marks

Solution Architecture:

Solution Architecture is the practice of designing the specific technical configuration that solves a business problem. If the Business Analyst identifies *what* needs to be done, the Solution Architect defines *how* the components (software, hardware, and data) will fit together to make it happen.

Think of it as the "blueprinting" phase that sits between the abstract business idea and the concrete code.

---

1. The Core Components

A robust Solution Architecture typically addresses four key domains:

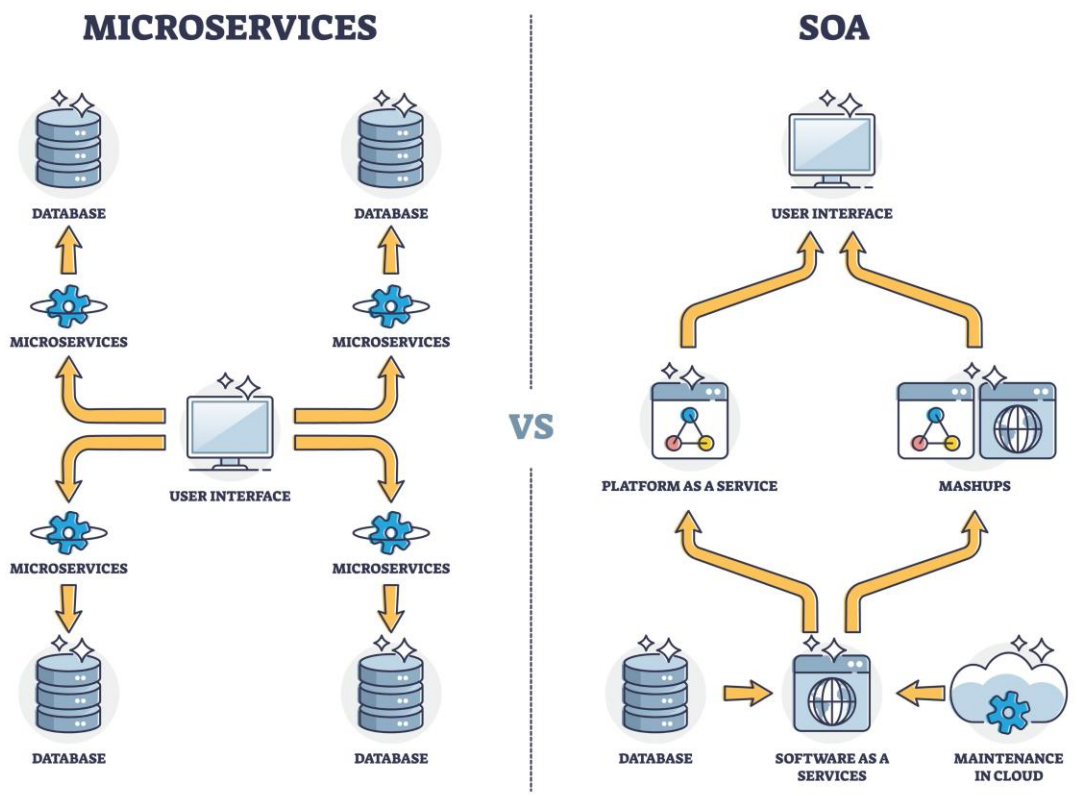
- **Application Architecture: Defining the software stack (e.g., Microservices vs. Monolith) and how different services communicate (APIs, Message Queues).**
- **Data Architecture: Determining where data is stored (SQL vs. NoSQL), how it flows, and how it stays secure.**
- **Infrastructure Architecture: Deciding where the solution lives—on-premise servers, Public Cloud (AWS/Azure/GCP), or a Hybrid setup.**
- **Security Architecture: Implementing identity management, encryption standards, and compliance (like GDPR or HIPAA).**

---

2. Common Architectural Patterns

Choosing the right pattern is the most critical decision an architect makes. Here are the "big three" used in modern development:

Pattern	Best For...	Pros	Cons
Monolithic	Small teams, simple apps.	Easy to deploy and test.	Hard to scale; "spaghetti" code risk.
Microservices	Large, complex systems.	Independent scaling; tech flexibility.	High operational complexity.
Serverless	Event-driven tasks, erratic traffic.	Low cost; no server management.	Vendor lock-in; cold start latency.



---

3. The Solution Architect's Workflow

The process isn't just about drawing diagrams; it's about balancing trade-offs.

1. **Analyze Requirements: Review the Functional and Non-Functional requirements (the ones we discussed earlier).**
2. **Select Tech Stack: Pick the languages and frameworks (e.g., React for frontend, Python/Node.js for backend).**
3. **Design for "The -ilities": Ensure the design is Scalable (can grow), Available (doesn't crash), and Maintainable (easy to fix).**
4. **Gap Analysis: Compare the "As-Is" state (current system) with the "To-Be" state (the goal) to see what needs to be built from scratch.**

---

4. Example Diagram

A typical high-level architecture diagram shows how a user interacts with the system through a Load Balancer, which routes traffic to various services and databases.